BIG DATA TECHNOLOGY LAB (BDTL) · PRACTICAL REPORT

# Bucket 7 · Apache Pig Experiments 1 and 4

Submitted by

## Adesh Patankar

| Experiment | Title and learning objective |
|---|---|
| Experiment 1 | **Load and Inspect Data in Pig** Load a sample text file into Pig using LOAD command. Use DUMP and DESCRIBE to view schema and structure. |
| Experiment 4 | **Store Results into HDFS** Save the final word count output back to HDFS using the STORE command |

Platform: Apache Pig 0.18.0 on Apache Hadoop 3.3.6 (pseudo-distributed HDFS + YARN), Ubuntu 24.04 LTS. All execution evidence in this report consists of screenshots of the actual terminal session in which the experiments were run.

# Contents


| | Section |
|---|---|
| **Experiment 1** | **Load and Inspect Data in Pig** |
| | A. Experiment Details<br>B. Aim<br>C. Theory<br>D. Requirements<br>E. Procedure and Implementation<br>F. Execution Screenshots<br>G. Output and Results<br>H. Result<br>I. Observations and Troubleshooting |
| **Experiment 4** | **Store Results into HDFS** |
| | A. Experiment Details<br>B. Aim<br>C. Theory<br>D. Requirements<br>E. Procedure and Implementation<br>F. Execution Screenshots<br>G. Output and Results<br>H. Result<br>I. Observations and Troubleshooting |

# Bucket 7 – Experiment 1: Load and Inspect Data in Pig

## A. Experiment Details

| Field | Details |
|---|---|
| **Bucket** | 7 |
| **Sub-experiment No.** | 1 |
| **Title** | Load and Inspect Data in Pig |
| **Description / Learning Objective (bucket list)** | Load a sample text file into Pig using LOAD command. Use DUMP and DESCRIBE to view schema and structure. |
| **Student** | Adesh Patankar |
| **Subject** | Big Data Technology Lab (BDTL) |
| **Tool / Technology** | Apache Pig 0.18.0 (Pig Latin) on Apache Hadoop 3.3.6 |

## B. Aim

To load a sample text file stored in HDFS into Apache Pig using the `LOAD` operator, and to inspect the resulting relation with `DESCRIBE` (schema) and `DUMP` (data), with and without a declared schema.

## C. Theory

### C.1 Apache Pig

Apache Pig is a high-level platform on Hadoop for analysing large datasets as data flows. Scripts are written in **Pig Latin**; the Pig engine parses a script into a logical plan, optimises it, compiles it into MapReduce jobs and submits them to YARN, reading input from and writing output to HDFS. A **relation** is a bag of **tuples**; each tuple is an ordered set of **fields** (atoms such as `chararray`, `int`, `long`). Statements are evaluated lazily: `LOAD`, `FOREACH`, `GROUP` only build the plan, and jobs are launched when `DUMP` or `STORE` needs a result. In this experiment Pig runs in **MapReduce mode** (`pig -x mapreduce`).

### C.2 LOAD, DESCRIBE, DUMP, LIMIT

| Operator | Syntax | Purpose |
|---|---|---|
| LOAD | `r = LOAD 'path' [USING loader] [AS (schema)];` | Creates a relation from a file in HDFS. The load function (e.g. `PigStorage(delimiter)`, `TextLoader()`) splits each record into fields; the optional `AS` clause names and types the fields. |
| DESCRIBE | `DESCRIBE r;` | Prints the schema of a relation without running a job, e.g. `lines: {line: chararray}`. If no schema was declared Pig prints *Schema for r unknown*. |
| DUMP | `DUMP r;` | Executes the plan (a MapReduce job) and prints every tuple of the relation on the console. |
| LIMIT | `l = LIMIT r n;` | Keeps only n tuples – useful for inspecting a sample of a large dataset. |

Here `PigStorage('\n')` uses the newline as the field delimiter, so each line of the text file is loaded as a single `chararray` field. Without a schema, fields can only be addressed by position (`$0`, `$1`, …) and are typed `bytearray`.

## D. Requirements

| Item | Used in this experiment |
|---|---|
| **Host machine** | Apple MacBook Air (Apple Silicon, arm64) running macOS |
| **Virtual machine** | Multipass VM `bigdata` – Ubuntu 24.04 LTS (arm64), 2 vCPU, 4 GB RAM, 2 GB swap |
| **Java** | OpenJDK 8 |
| **Hadoop** | Apache Hadoop 3.3.6, pseudo-distributed: HDFS (`hdfs://localhost:9000`), YARN and the MapReduce JobHistory server |
| **Pig** | Apache Pig 0.18.0 (Hadoop 3 build), run in **MapReduce mode**: `pig -x mapreduce -f <script>.pig`; `PIG_HOME`, `PATH` and `PIG_CLASSPATH=$HADOOP_HOME/etc/hadoop` set in `~/.bashrc` |
| **Logging** | A custom `log4j.properties` (passed with `-4`) keeps Pig's job progress, job statistics and errors on the console and hides routine INFO messages |
| **Dataset** | `sample.txt` – 6 lines of text (46 words), uploaded to `/bdt/bucket7/sample.txt` in HDFS |
| **Runner** | `bdt/bucket7/run_bucket7.sh` – prints each Pig script (`cat`) and then runs it with `pig -x mapreduce -4 ~/bdt/pig/log4j.properties -f <script>` |

## E. Procedure and Implementation

### Step 1 – Prepare the input in HDFS

bucket7/data/sample.txt

```
Hadoop is an open source framework for big data
HDFS stores big data across many machines
MapReduce processes data in parallel on Hadoop
Pig is a high level platform on top of Hadoop
```

```
Pig Latin makes data processing easy
Hive and Pig both run on Hadoop
```

Shell commands

```
pig -version
jps
cat data/sample.txt
hdfs dfs -mkdir -p /bdt/bucket7
hdfs dfs -put -f data/sample.txt /bdt/bucket7/
hdfs dfs -ls /bdt/bucket7
hdfs dfs -cat /bdt/bucket7/sample.txt
```

## Step 2 – Load and inspect the data

bucket7/pig/01_load_inspect.pig

```
-- Bucket 7 | Sub-Experiment 1 : Load and inspect data in Pig
-- (a) LOAD with a schema: every line of the file becomes one tuple with one chararray field
lines = LOAD '/bdt/bucket7/sample.txt' USING PigStorage('\n') AS (line:chararray);
DESCRIBE lines;
DUMP lines;

-- (b) LOAD without a schema: fields are positional ($0, $1 ...) and their type is bytearray
raw = LOAD '/bdt/bucket7/sample.txt';
DESCRIBE raw;

-- (c) inspect only the first 3 records
first3 = LIMIT lines 3;
DESCRIBE first3;
DUMP first3;
```

Command

```
pig -x mapreduce -4 ~/bdt/pig/log4j.properties -f pig/01_load_inspect.pig
```

## F. Execution Screenshots

Screenshots of the Ubuntu VM terminal (`ubuntu@bigdata`) while `run_bucket7.sh` was executing. Only empty margins were trimmed.

```
BUCKET 7 : Apache Pig - Load, Count and Store Word Frequencies
Part 1 : Sub-Exp 1 (setup): environment check, sample text file and upload to HDFS

$ pig -version
Apache Pig version 0.18.0 (r1926484)
compiled Jun 16 2025, 10:38:21
$ jps
1696 NameNode
3377 JobHistoryServer
2483 ResourceManager
48532 Jps
1916 DataNode
2157 SecondaryNameNode
2687 NodeManager
$ cat data/sample.txt
Hadoop is an open source framework for big data
HDFS stores big data across many machines
MapReduce processes data in parallel on Hadoop
Pig is a high level platform on top of Hadoop
Pig Latin makes data processing easy
Hive and Pig both run on Hadoop
$ hdfs dfs -mkdir -p /bdt/bucket7
$ hdfs dfs -put -f data/sample.txt /bdt/bucket7/
$ hdfs dfs -ls /bdt/bucket7
Found 1 items
-rw-r--r--   1 ubuntu supergroup        252 2026-10-25 23:25 /bdt/bucket7/sample.txt
$ hdfs dfs -cat /bdt/bucket7/sample.txt
Hadoop is an open source framework for big data
HDFS stores big data across many machines
MapReduce processes data in parallel on Hadoop
Pig is a high level platform on top of Hadoop
Pig Latin makes data processing easy
Hive and Pig both run on Hadoop
```

**Figure 1.1:** Environment check, sample text file and upload to HDFS

`pig -version` reports Apache Pig 0.18.0 and `jps` lists NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager and JobHistoryServer. `sample.txt` (6 lines) is copied to `/bdt/bucket7/`; `hdfs dfs -ls` shows the 252-byte file and `hdfs dfs -cat` prints its contents from HDFS.

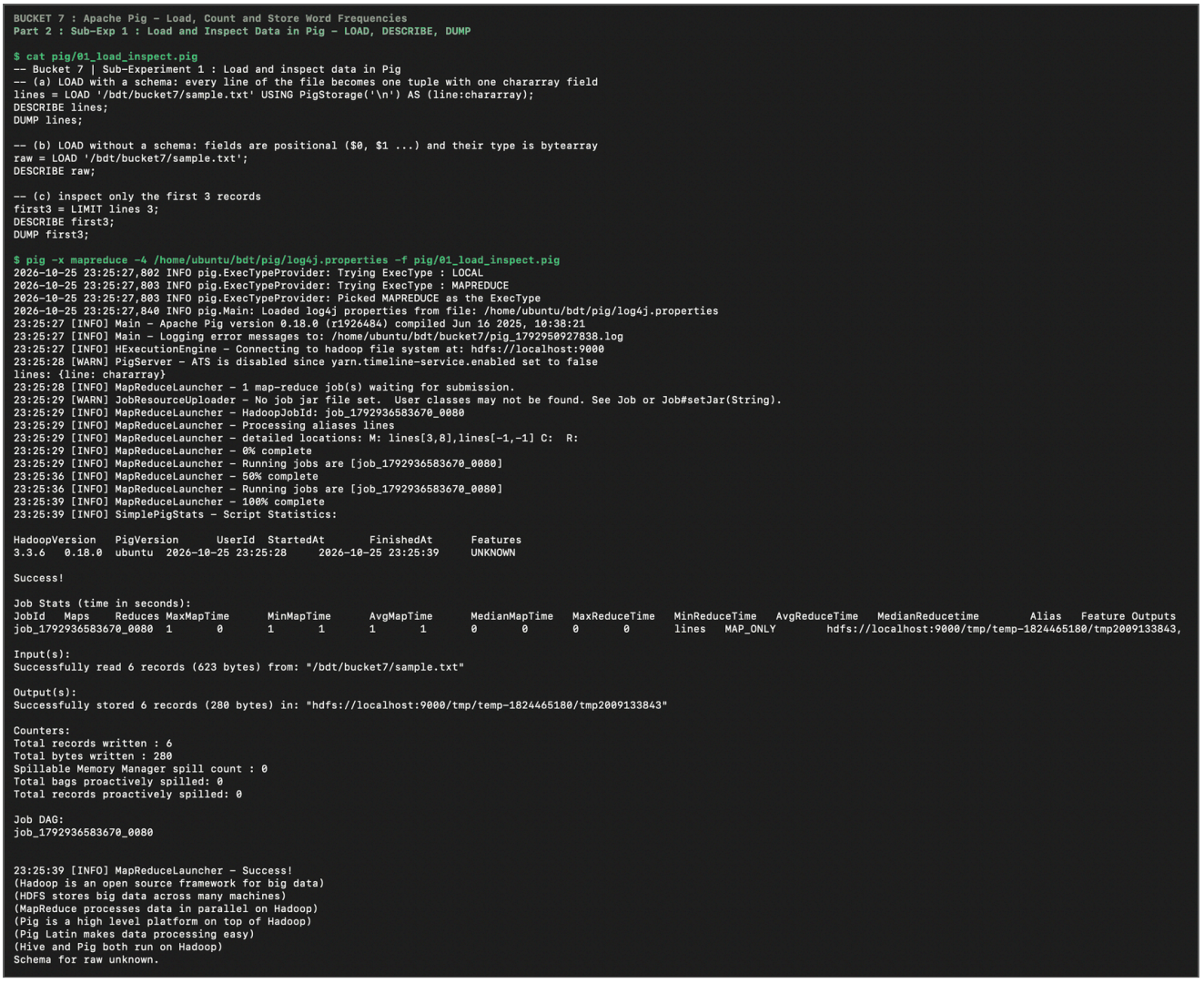



**Figure 1.2:** LOAD, DESCRIBE and DUMP

The script is printed and run in MapReduce mode. `DESCRIBE lines` prints `lines: {line: chararray}`. The DUMP runs as map-only job `job_..._0080` (*Successfully read 6 records*, *Success!*) and prints the 6 lines as tuples. `DESCRIBE raw` prints *Schema for raw unknown.* because `raw` was loaded without an `AS` clause.

## G. Output and Results

| Statement | Output |
|---|---|
| DESCRIBE lines | `lines: {line: chararray}` |
| DUMP lines | 6 tuples, e.g. `(Hadoop is an open source framework for big data)` … `(Hive and Pig both run on Hadoop)` |
| DESCRIBE raw | `Schema for raw unknown.` |
| Job statistics | 1 map, 0 reduces (MAP_ONLY); 6 records read, 6 records written |

The last part of the script (`LIMIT lines 3`, `DESCRIBE first3`, `DUMP first3`) ran after the point captured in Figure 1.2; its output is not shown in the screenshots and is therefore not reported here.

## H. Result

> The objective was achieved. The sample text file was loaded from HDFS into Pig with `LOAD`, its schema was displayed with `DESCRIBE` (`{line: chararray}`, and *unknown* when no schema is given), and its contents were displayed with `DUMP` – all 6 lines, matching the input file.

## I. Observations and Troubleshooting

### Observations

- `DESCRIBE` returned immediately without launching a job; `DUMP` launched a MapReduce job (~11 s).
- Loading and dumping needs no reduce phase, so Pig ran a map-only job.
- Declaring a schema with `AS` lets fields be referred to by name and type; without it Pig cannot describe the relation.

### Setup notes

- Apache Pig 0.18.0 (Hadoop 3 build) is used; `PIG_HOME`, `PATH` and `PIG_CLASSPATH=$HADOOP_HOME/etc/hadoop` are set in `~/.bashrc`.
- The MapReduce JobHistory server must be running (`mapred --daemon start historyserver`) because Pig queries it for job statistics after every job.
- The warnings *No job jar file set*, *ATS is disabled* and *SchemaTupleBackend has already been initialized* are harmless in this pseudo-distributed setup.
- The run was executed once and completed without errors; no troubleshooting was required for this experiment.

# Bucket 7 – Experiment 4: Store Results into HDFS

## A. Experiment Details

| Field | Details |
|---|---|
| **Bucket** | 7 |
| **Sub-experiment No.** | 4 |
| **Title** | Store Results into HDFS |
| **Description / Learning Objective (bucket list)** | Save the final word count output back to HDFS using the STORE command (as given in the bucket list) |
| **Student** | Adesh Patankar |
| **Subject** | Big Data Technology Lab (BDTL) |
| **Tool / Technology** | Apache Pig 0.18.0 (Pig Latin) on Apache Hadoop 3.3.6 |

## B. Aim

To compute the word count of a text file in Pig and save the final result back into HDFS with the `STORE` command, then verify the stored output from Pig and from the HDFS shell.

## C. Theory

### C.1 Apache Pig

Apache Pig is a high-level platform on Hadoop for analysing large datasets as data flows. Scripts are written in **Pig Latin**; the Pig engine parses a script into a logical plan, optimises it, compiles it into MapReduce jobs and submits them to YARN, reading input from and writing output to HDFS. A **relation** is a bag of **tuples**; each tuple is an ordered set of **fields** (atoms such as `chararray`, `int`, `long`). Statements are evaluated lazily: `LOAD`, `FOREACH`, `GROUP` only build the plan, and jobs are launched when `DUMP` or `STORE` needs a result. In this experiment Pig runs in **MapReduce mode** (`pig -x mapreduce`).

### C.2 The STORE operator

| Item | Description |
|---|---|
| Syntax | `STORE relation INTO 'hdfs_directory' [USING store_function];` |
| Output | STORE writes to a **directory**, not a single file. Each reducer writes a `part-r-NNNNN` file (`part-m-NNNNN` for map-only jobs) and a zero-byte `_SUCCESS` marker is written when the job completes. |
| Store function | `PigStorage(',')` writes each tuple as one text line with fields separated by commas (default delimiter is tab). |
| Existing output | STORE fails if the output directory already exists, so the script first runs `rmf` (remove, no error if missing). |
| DUMP vs STORE | DUMP prints results on the console (stored in a temporary HDFS location); STORE persists them at a chosen HDFS path for later use. |
| fs commands | Inside a Pig script `fs -ls`, `fs -cat` run HDFS shell commands. |

The word count itself is computed with TOKENIZE/FLATTEN, GROUP BY word and FOREACH ... GENERATE COUNT(), then sorted with ORDER BY.

## D. Requirements

| Item | Used in this experiment |
|---|---|
| **Host machine** | Apple MacBook Air (Apple Silicon, arm64) running macOS |
| **Virtual machine** | Multipass VM `bigdata` – Ubuntu 24.04 LTS (arm64), 2 vCPU, 4 GB RAM, 2 GB swap |
| **Java** | OpenJDK 8 |
| **Hadoop** | Apache Hadoop 3.3.6, pseudo-distributed: HDFS (`hdfs://localhost:9000`), YARN and the MapReduce JobHistory server |
| **Pig** | Apache Pig 0.18.0 (Hadoop 3 build), run in **MapReduce mode**: `pig -x mapreduce -f <script>.pig`; `PIG_HOME`, `PATH` and `PIG_CLASSPATH=$HADOOP_HOME/etc/hadoop` set in `~/.bashrc` |
| **Logging** | A custom `log4j.properties` (passed with `-4`) keeps Pig's job progress, job statistics and errors on the console and hides routine INFO messages |
| **Dataset** | `sample.txt` – 6 lines of text (46 words), uploaded to `/bdt/bucket7/sample.txt` in HDFS |
| **Runner** | `bdt/bucket7/run_bucket7.sh` – prints each Pig script (`cat`) and then runs it with `pig -x mapreduce -4 ~/bdt/pig/log4j.properties -f <script>` |
| **Output location** | `/bdt/bucket7/wordcount_output` (HDFS) and `~/bdt/bucket7/output_local` (local copy) |

# E. Procedure and Implementation

## Step 1 – Prepare the input in HDFS

bucket7/data/sample.txt

```
Hadoop is an open source framework for big data
HDFS stores big data across many machines
MapReduce processes data in parallel on Hadoop
Pig is a high level platform on top of Hadoop
Pig Latin makes data processing easy
Hive and Pig both run on Hadoop
```

Shell commands

```
pig -version
jps
cat data/sample.txt
hdfs dfs -mkdir -p /bdt/bucket7
hdfs dfs -put -f data/sample.txt /bdt/bucket7/
hdfs dfs -ls /bdt/bucket7
hdfs dfs -cat /bdt/bucket7/sample.txt
```

## Step 2 – Compute the word count and STORE it

bucket7/pig/04_store_hdfs.pig

```
-- Bucket 7 | Sub-Experiment 4 : Store the word-count result back into HDFS
lines   = LOAD '/bdt/bucket7/sample.txt' USING PigStorage('\n') AS (line:chararray);
words   = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) AS word;
grouped = GROUP words BY word;
counts  = FOREACH grouped GENERATE group AS word, COUNT(words) AS frequency;
result  = ORDER counts BY frequency DESC, word ASC;

-- remove an old output directory (STORE fails if the directory already exists)
rmf /bdt/bucket7/wordcount_output
STORE result INTO '/bdt/bucket7/wordcount_output' USING PigStorage(',');

-- verify from inside Pig
fs -ls /bdt/bucket7/wordcount_output
fs -cat /bdt/bucket7/wordcount_output/part*

-- read the stored result back into Pig to confirm it is usable
stored = LOAD '/bdt/bucket7/wordcount_output' USING PigStorage(',') AS (word:chararray,
frequency:long);
top5   = LIMIT stored 5;
DUMP top5;
```

Command

```
pig -x mapreduce -4 ~/bdt/pig/log4j.properties -f pig/04_store_hdfs.pig
```

## Step 3 – Verify the stored output from the HDFS shell

Shell commands

```
hdfs dfs -ls -R /bdt/bucket7
hdfs dfs -cat /bdt/bucket7/wordcount_output/part-r-00000
hdfs dfs -get /bdt/bucket7/wordcount_output output_local
ls -l output_local
wc -l output_local/part-r-00000
```

## F. Execution Screenshots

Screenshots of the Ubuntu VM terminal (`ubuntu@bigdata`) while `run_bucket7.sh` was executing. Only empty margins were trimmed.

```
BUCKET 7 : Apache Pig - Load, Count and Store Word Frequencies
Part 1 : Sub-Exp 1 (setup): environment check, sample text file and upload to HDFS

$ pig -version
Apache Pig version 0.18.0 (r1926484)
compiled Jun 16 2025, 10:38:21
$ jps
1696 NameNode
3377 JobHistoryServer
2483 ResourceManager
48532 Jps
1916 DataNode
2157 SecondaryNameNode
2687 NodeManager
$ cat data/sample.txt
Hadoop is an open source framework for big data
HDFS stores big data across many machines
MapReduce processes data in parallel on Hadoop
Pig is a high level platform on top of Hadoop
Pig Latin makes data processing easy
Hive and Pig both run on Hadoop
$ hdfs dfs -mkdir -p /bdt/bucket7
$ hdfs dfs -put -f data/sample.txt /bdt/bucket7/
$ hdfs dfs -ls /bdt/bucket7
Found 1 items
-rw-r--r--   1 ubuntu supergroup        252 2026-10-25 23:25 /bdt/bucket7/sample.txt
$ hdfs dfs -cat /bdt/bucket7/sample.txt
Hadoop is an open source framework for big data
HDFS stores big data across many machines
MapReduce processes data in parallel on Hadoop
Pig is a high level platform on top of Hadoop
Pig Latin makes data processing easy
Hive and Pig both run on Hadoop
```

**Figure 4.1:** Environment check, sample text file and upload to HDFS

`pig -version` reports Apache Pig 0.18.0 and `jps` lists NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager and JobHistoryServer. `sample.txt` (6 lines) is copied to `/bdt/bucket7/`; `hdfs dfs -ls` shows the 252-byte file and `hdfs dfs -cat` prints its contents from HDFS.

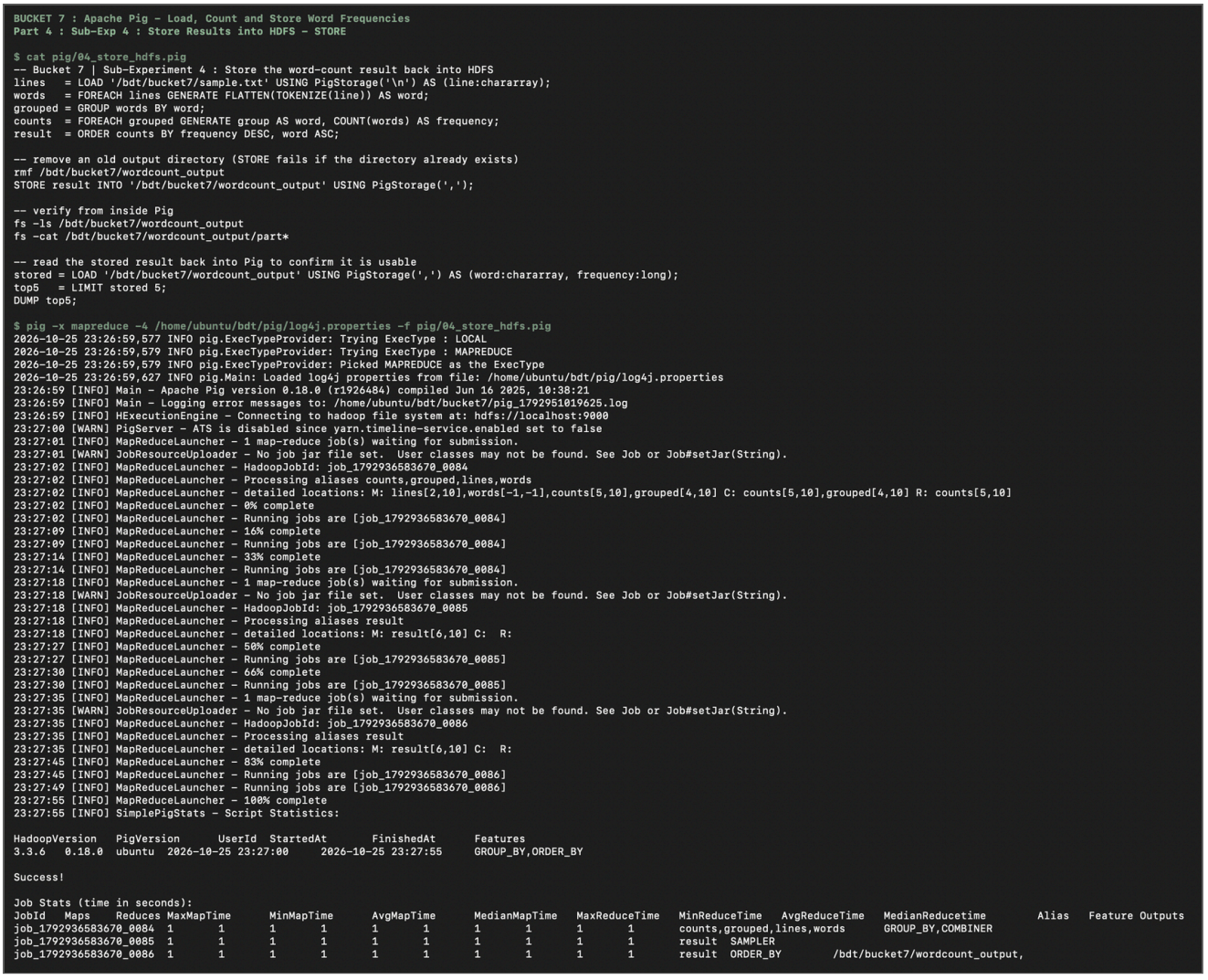



**Figure 4.2:** STORE script and MapReduce jobs

The script is printed and executed. Pig compiles the word count and the STORE into three jobs (`_0084` GROUP_BY+COMBINER, `_0085` SAMPLER, `_0086` ORDER_BY); the job statistics show the output of the last job is `/bdt/bucket7/wordcount_output`.

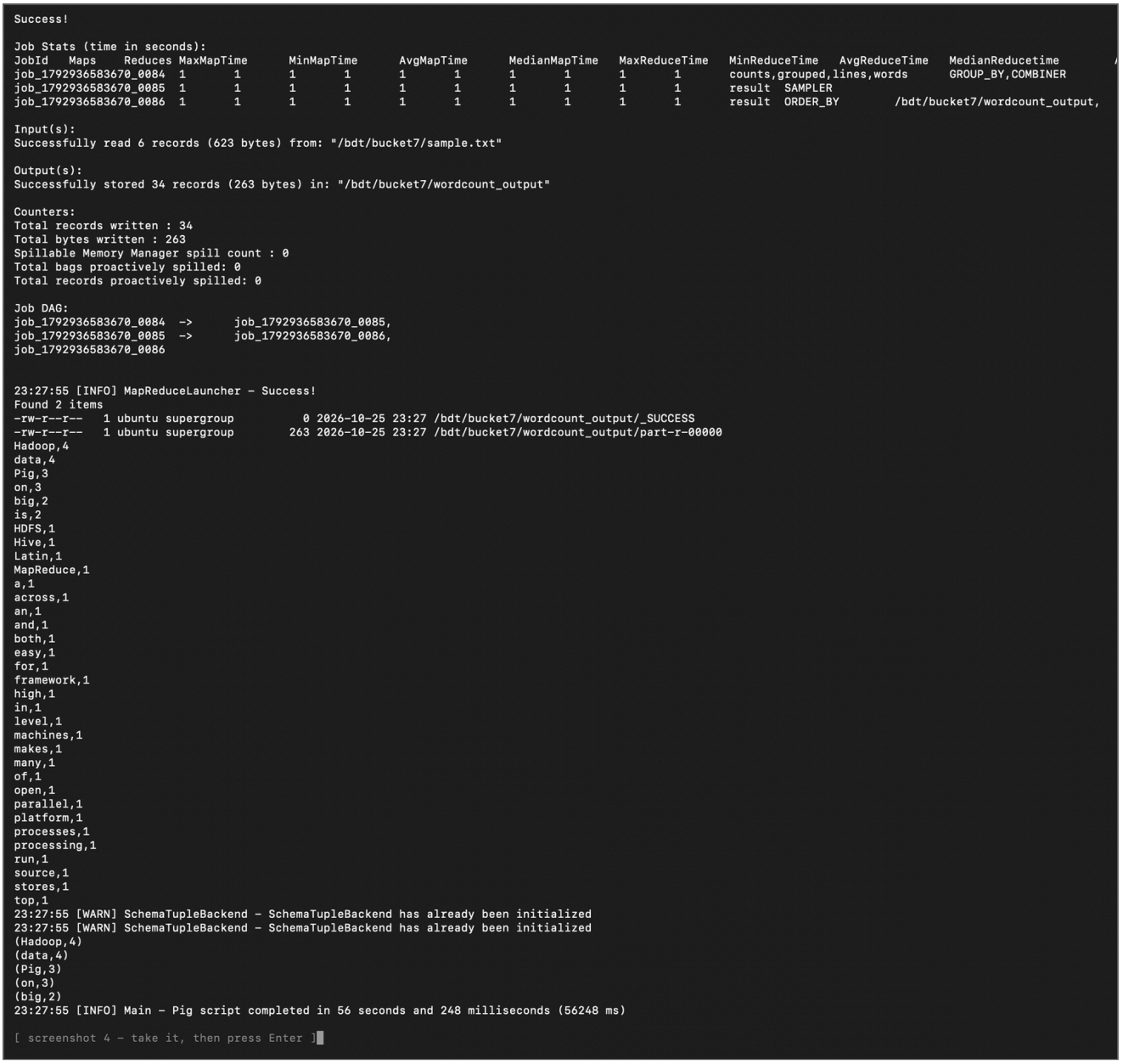



**Figure 4.3:** Result stored in HDFS and read back

*Successfully stored 34 records (263 bytes) in "/bdt/bucket7/wordcount_output"*. `fs -ls` shows `_SUCCESS` and the 263-byte `part-r-00000`; `fs -cat` prints the comma-separated result (Hadoop,4 data,4 Pig,3 on,3 big,2 …). Loading the stored directory back and dumping 5 tuples returns (Hadoop,4) (data,4) (Pig,3) (on,3) (big,2).

```
BUCKET 7 : Apache Pig - Load, Count and Store Word Frequencies
Part 5 : Sub-Exp 4 : Verify the stored output from the HDFS shell

$ hdfs dfs -ls -R /bdt/bucket7
-rw-r--r--   1 ubuntu supergroup        252 2026-10-25 23:25 /bdt/bucket7/sample.txt
drwxr-xr-x   - ubuntu supergroup          0 2026-10-25 23:27 /bdt/bucket7/wordcount_output
-rw-r--r--   1 ubuntu supergroup          0 2026-10-25 23:27 /bdt/bucket7/wordcount_output/_SUCCESS
-rw-r--r--   1 ubuntu supergroup        263 2026-10-25 23:27 /bdt/bucket7/wordcount_output/part-r-00000
$ hdfs dfs -cat /bdt/bucket7/wordcount_output/part-r-00000
Hadoop,4
data,4
Pig,3
on,3
big,2
is,2
HDFS,1
Hive,1
Latin,1
MapReduce,1
a,1
across,1
an,1
and,1
both,1
easy,1
for,1
framework,1
high,1
in,1
level,1
machines,1
makes,1
many,1
of,1
open,1
parallel,1
platform,1
processes,1
processing,1
run,1
source,1
stores,1
top,1
$ hdfs dfs -get /bdt/bucket7/wordcount_output output_local
$ ls -l output_local
total 4
-rw-r--r-- 1 ubuntu ubuntu   0 Oct 25 23:28 _SUCCESS
-rw-r--r-- 1 ubuntu ubuntu 263 Oct 25 23:28 part-r-00000
$ wc -l output_local/part-r-00000
34 output_local/part-r-00000

[ screenshot 5 - take it, then press Enter ]
```

**Figure 4.4:** Verification from the HDFS shell

`hdfs dfs -ls -R` shows the input file and the output directory; `hdfs dfs -cat` prints the stored file; `hdfs dfs -get` copies it to the local file system, where `ls -l` shows the same 263-byte `part-r-00000` and `wc -l` confirms 34 lines.

## G. Output and Results

| Check | Result |
|---|---|
| Output directory | `/bdt/bucket7/wordcount_output` containing `_SUCCESS` (0 B) and `part-r-00000` (263 B) |
| Records stored | 34 (one per distinct word) |
| Format | `word,frequency` per line, sorted by frequency, e.g. `Hadoop,4`, `data,4`, `Pig,3` |
| Read back into Pig | `(Hadoop,4) (data,4) (Pig,3) (on,3) (big,2)` |
| Local copy | `output_local/part-r-00000`, 263 B, 34 lines |

The stored file is identical whether viewed from Pig, from the HDFS shell or after copying it locally, which shows that STORE persisted the final word count in HDFS in a reusable form.

## H. Result

> The objective was achieved. The final word-count result (34 distinct words) was saved back to HDFS with `STORE ... INTO '/bdt/bucket7/wordcount_output' USING PigStorage(',')`. The output directory, its `_SUCCESS` marker and `part-r-00000` file were verified with Pig `fs` commands, by loading the result back into Pig, and with `hdfs dfs -ls/-cat/-get`.

## I. Observations and Troubleshooting

### Observations

- The output file is named `part-r-00000` because the final job (ORDER BY) had one reducer.
- The stored file (263 B) is smaller than the DUMP's temporary output (347 B) because PigStorage(',') writes plain text without Pig's internal serialisation.
- `rmf` before STORE makes the script re-runnable; without it a second run fails because the output directory exists.

### Setup notes

- Apache Pig 0.18.0 (Hadoop 3 build) is used; `PIG_HOME`, `PATH` and `PIG_CLASSPATH=$HADOOP_HOME/etc/hadoop` are set in `~/.bashrc`.
- The MapReduce JobHistory server must be running (`mapred --daemon start historyserver`) because Pig queries it for job statistics after every job.
- The warnings *No job jar file set*, *ATS is disabled* and *SchemaTupleBackend has already been initialized* are harmless in this pseudo-distributed setup.
- The run was executed once and completed without errors; no troubleshooting was required for this experiment.